Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

# ЗВІТ

з лабораторної роботи № 1

з дисципліни «Теорія прийняття рішень»

**Тема: «Розподіли випадкових величин — неперервні і дискретні»**

Варіант № 1

Виконав:
студент групи ФЕІ-34с
Біжик Юрій Володимирович

Перевірила:
доцент Дуфанець Марта
Василівна

Львів — 2026

## 1 Мета роботи

Вивчити поняття розподілів дискретних та неперервних випадкових величин, реалізувати ввід параметрів і побудову графіків відповідних законів розподілу. Обчислити теоретичні характеристики та пояснити відмінність між імовірнісною масою, густиною та функцією розподілу.

## 2 Завдання та вихідні дані

Мій порядковий номер у журналі $N = 1$. За методичкою потрібно зобразити сім розподілів із параметрами, наведеними нижче. Програма приймає N через командний рядок або всі параметри через JSON.

| **Розподіл** | **Параметри для N=1** |
|---|---|
| Бернуллі | $p = \frac{1}{N+1} = 0.5$ |
| Біноміальний | $p = 0.5, n = N + 2 = 3$ |
| Пуассона | $\lambda = N + 10 = 11, k_{\max} = N + 5 = 6$ |
| Рівномірний | $a = -N = -1, b = N = 1$ |
| Нормальний | $\mu = N = 1, \sigma^2 = \frac{N}{2} = 0.5$ |
| Парето | $\alpha = x_0 = N = 1$ |
| Стьюдента | $\nu = N = 1$ |

Таблиця 1 — Параметри розподілів згідно з індивідуальним варіантом

Випадкова вибірка в цій роботі не потрібна: графіки побудовані безпосередньо за аналітичними PMF/PDF. Збережені числові дані є значеннями функцій на сітці, а не експериментальними частотами.

## 3 Теоретичні відомості

Для дискретної величини закон задається імовірнісною масою $p_k = P(X = k)$, сума всіх мас дорівнює 1. Дискретні значення зображаються окремими стовпцями, а не неперервною густиною.

Для неперервної величини густина $f(x)$ є невід'ємною і має інтеграл 1. Значення густини саме по собі не є імовірністю події $X = x$: для абсолютно неперервного розподілу $P(X = x) = 0$. Імовірність інтервалу визначається інтегралом густини.

В обох випадках $F(x) = P(X \le x)$. Для дискретних законів це правобічно неперервна ступінчаста функція; для наведених неперервних — неперервна інтегральна функція. У кожному рисунку верхня панель показує PMF або PDF синім кольором, нижня — CDF помаранчевим. Кожна панель має власну вертикальну шкалу.

За умови існування моментів $E(X) = \sum_k k p_k$ або $E(X) = \int_{-\infty}^{\infty} x f(x)\,\mathrm{d}x$, а дисперсія $D(X) = E\big((X - E(X))^2\big)$. Скінченність цих інтегралів потрібно обґрунтовувати для розподілів із важкими хвостами.

## 4 Програмна реалізація

Код написано мовою Python із використанням лише стандартної бібліотеки. `math.comb` використано для біноміальних коефіцієнтів, `math.lgamma` — для гамма-функції та стійкого обчислення мас Пуассона, `math.erf` — для нормальної CDF. Для Стьюдента при $\nu$=1 використано точну CDF через арктангенс; для інших додатних $\nu$ передбачено регуляризовану неповну бета-функцію з обчисленням ланцюгового дробу.

Для неперервних розподілів створено сітки з 801 базової точки. Біля розривів густини рівномірного та Парето додано близькі точки обабіч меж, щоб не згладжувати розриви помітними похилими відрізками. Для дискретних розподілів використовуються всі значення в заданому діапазоні. Маси Пуассона не перенормовуються після обрізання.

Програма зберігає параметри, моменти й значення PMF/PDF та CDF у `data/results.json`, а кожен розподіл — також в окремий CSV. `src/charts.py` створює самодостатні SVG з українськими підписами. `index.html` містить усі рисунки для зручного перегляду.

Виконано команду:

```
uv run python src/main.py --student-number 1 --input data/input.json
```

Числові результати й рисунки нижче отримано саме цим запуском.

## 5 Результати та аналіз розподілів

### 5.1 Бернуллі

$P(X = 0) = 1 - p$, $P(X = 1) = p$; для мого варіанта обидва значення дорівнюють 0.5. Сподівання $E(X) = p = 0.5$, дисперсія $D(X) = p(1 - p) = 0.25$. CDF дорівнює 0 для $x < 0$, 0.5 на відрізку $0 \le x < 1$ і 1 для x≥1.

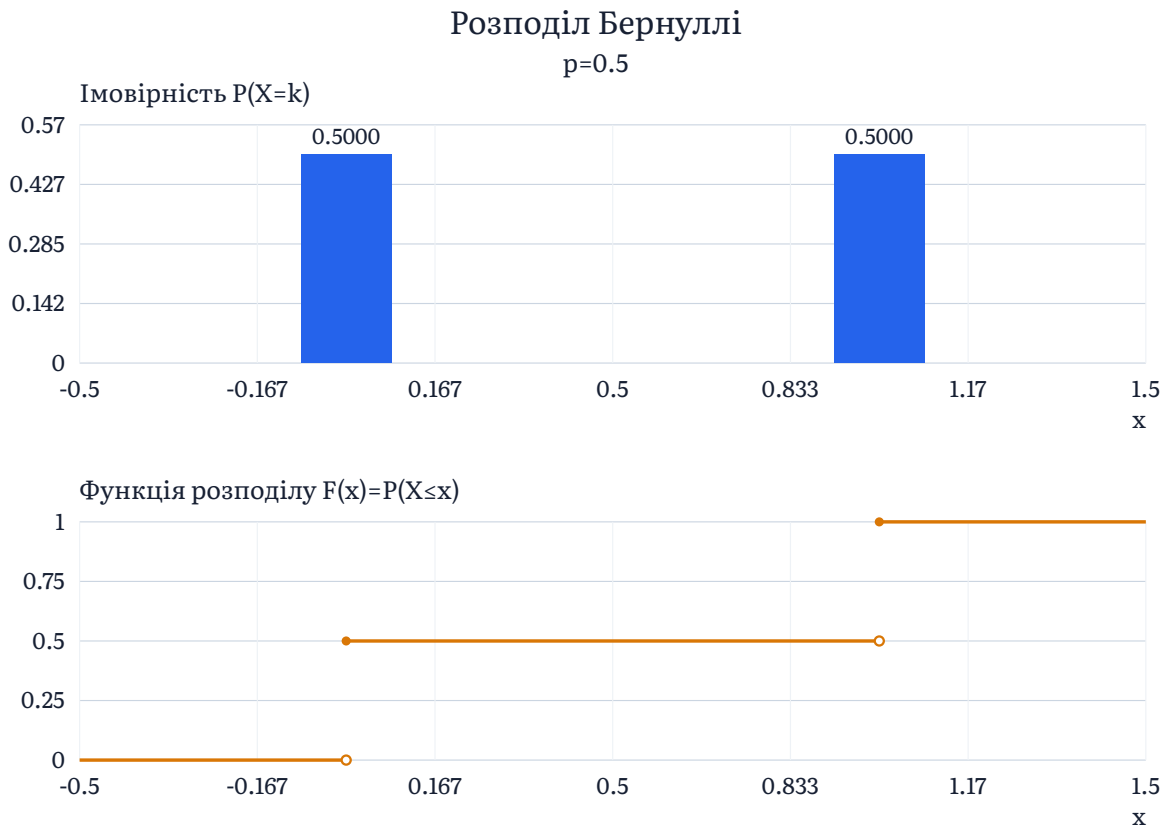



Рисунок 1 — Бернуллі: дві рівноймовірні події та ступінчаста CDF

### 5.2 Біноміальний розподіл

Біноміальна величина рахує кількість успіхів у n незалежних випробуваннях Бернуллі з незмінною імовірністю p:

$$P(X = k) = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}, \quad k = 0, \ldots, n. \tag{1}$$

Для n=3 і p=0.5 маси дорівнюють 0.125, 0.375, 0.375 та 0.125. Моди — 1 і 2, а симетрія відносно 1.5 узгоджується зі сподіванням $np = 1.5$. Дисперсія $np(1 - p) = 0.75$.

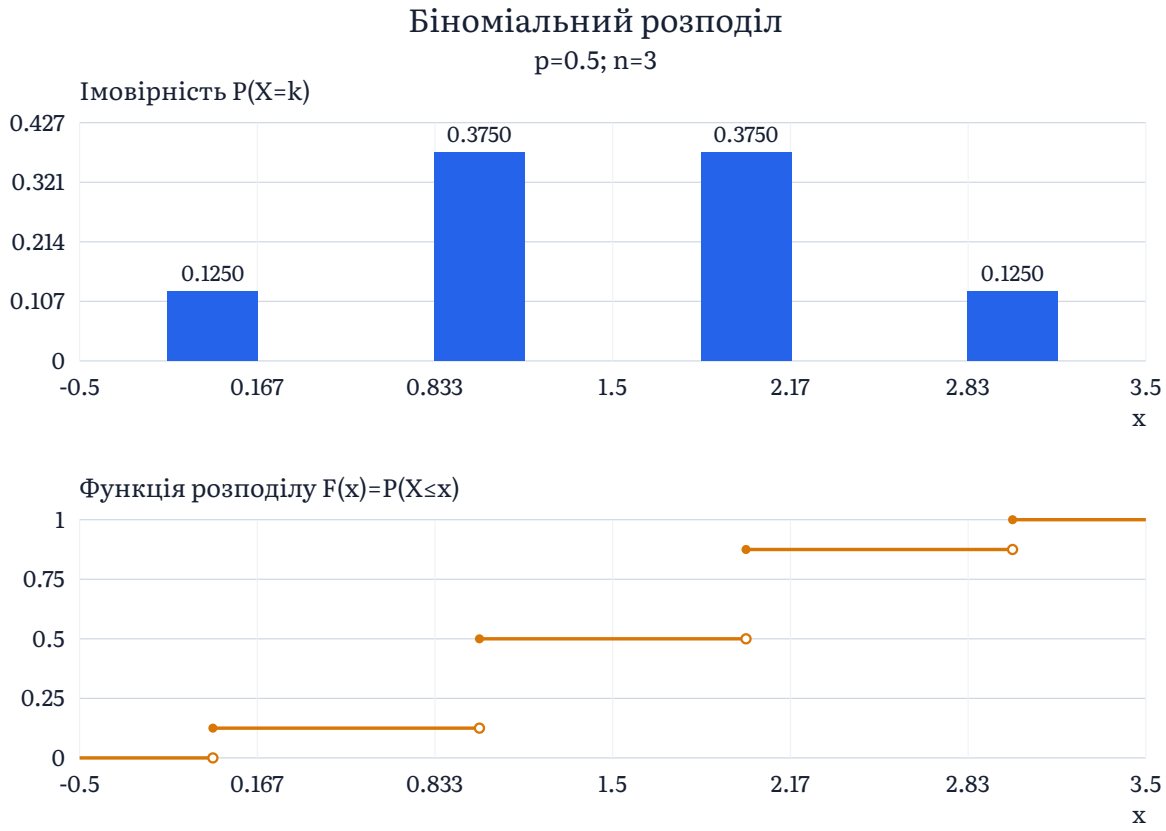



Рисунок 2 — Біноміальний розподіл із двома модами для n=3 та p=0.5

### 5.3 Розподіл Пуассона

$$P(X = k) = \frac{\lambda^k \exp(-\lambda)}{k!}, \quad k = 0, 1, \ldots. \tag{2}$$

Розподіл має нескінченну підтримку, $E(X) = D(X) = \lambda = 11$. Завдання обмежує рисунок значеннями k=0..6; це ліва частина розподілу, яка не включає жодної з його мод 10 та 11. Усі показані маси зростають із k, тому їхній максимум на рисунку не є глобальною модою.

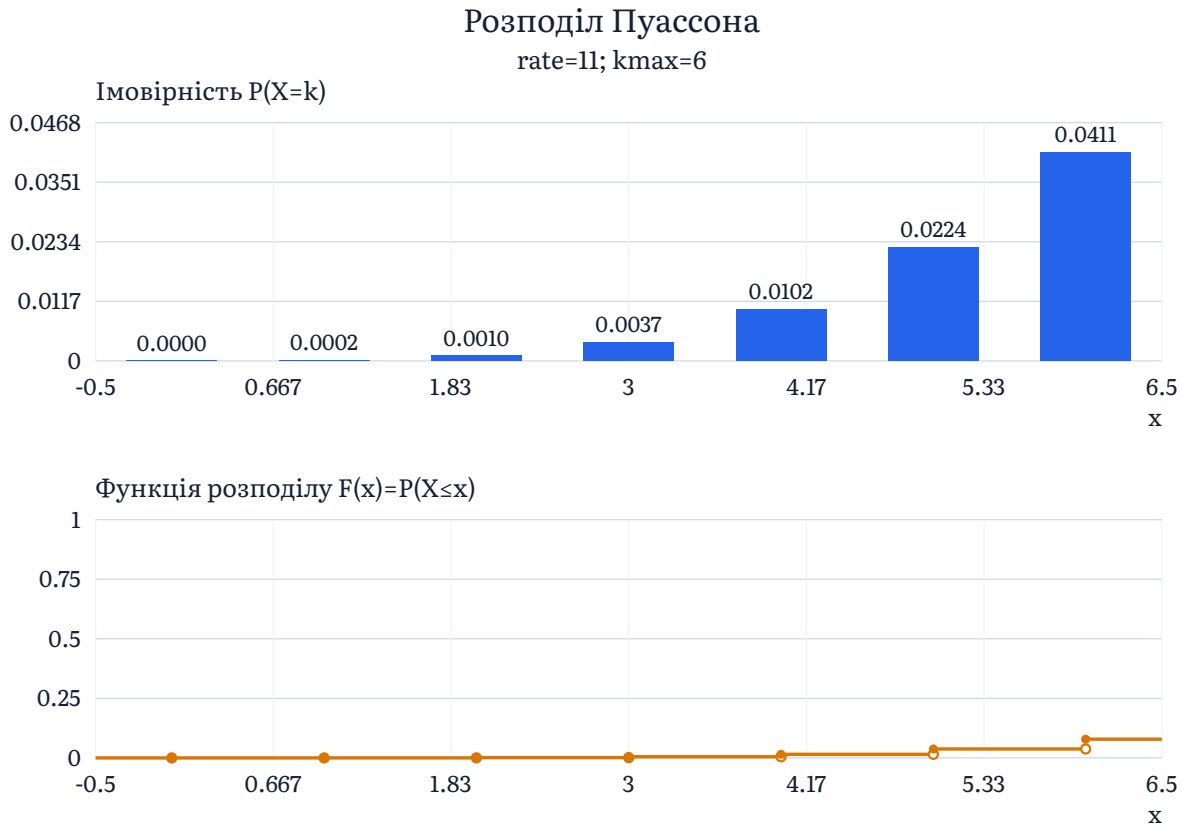



Рисунок 3 — Пуассон λ=11: лише заданий діапазон k=0..6 без перенормування

Сума показаних мас дорівнює 0.078614, тобто приблизно 7.8614%. За межею рисунка лишається 0.921386 імовірності, приблизно 92.1386%. Тому CDF у правій частині нижньої панелі не досягає одиниці. Обрізання графіка не змінює теоретичні моменти всього розподілу.

### 5.4 Рівномірний розподіл

На інтервалі від a до b густина стала: $f(x) = \frac{1}{b-a} = 0.5$; поза ним вона дорівнює нулю. Значення густини в окремих граничних точках не змінює інтеграли. CDF лінійно зростає від 0 до 1 на [−1,1]. Сподівання $\frac{a+b}{2} = 0$, дисперсія $\frac{(b-a)^2}{12} = \frac{1}{3}$.

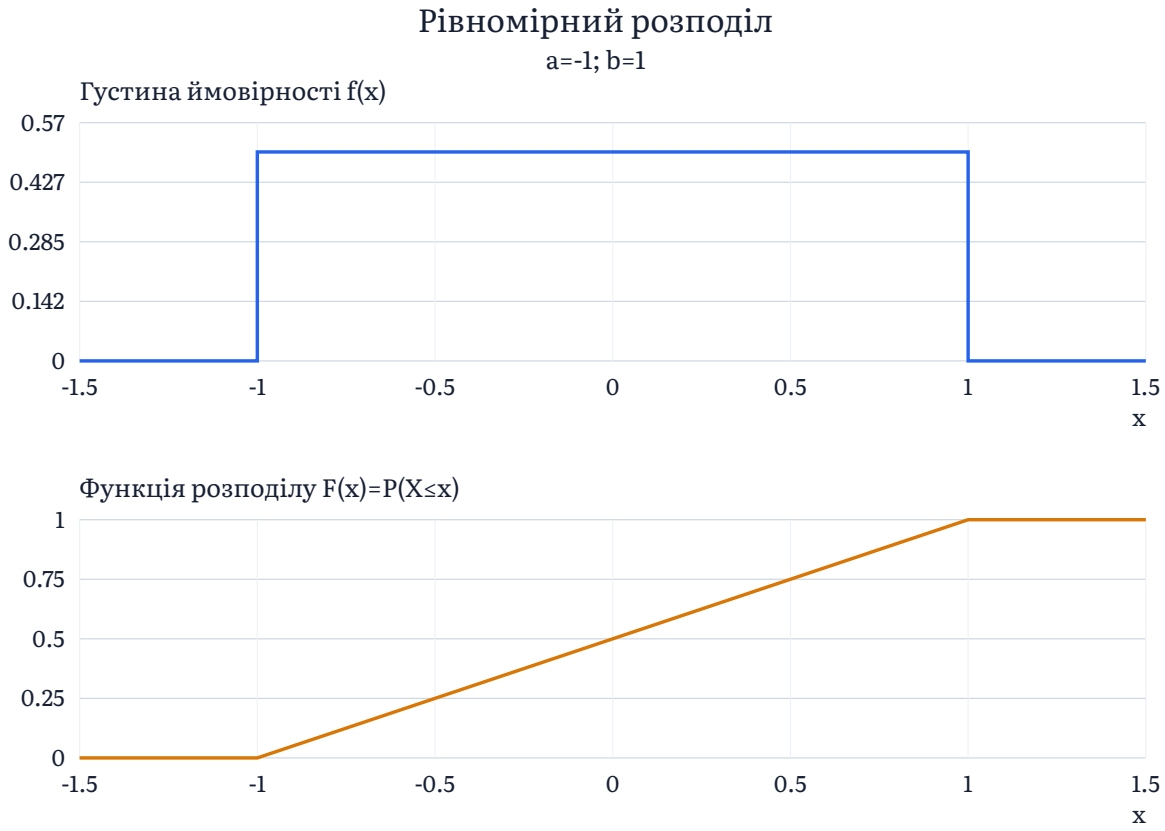



Рисунок 4 — Рівномірний розподіл на [−1,1]: стала густина та лінійна CDF

## 5.5 Нормальний розподіл

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{3}$$

Середнє mu=1, дисперсія $\sigma^2 = 0.5$, тому стандартне відхилення $\sigma = \sqrt{0.5} \approx 0.707107$. Важливо не підставляти 0.5 як стандартне відхилення: в умові задано саме дисперсію. Густина симетрична відносно 1, де вона має максимум, а CDF у цій точці дорівнює 0.5.

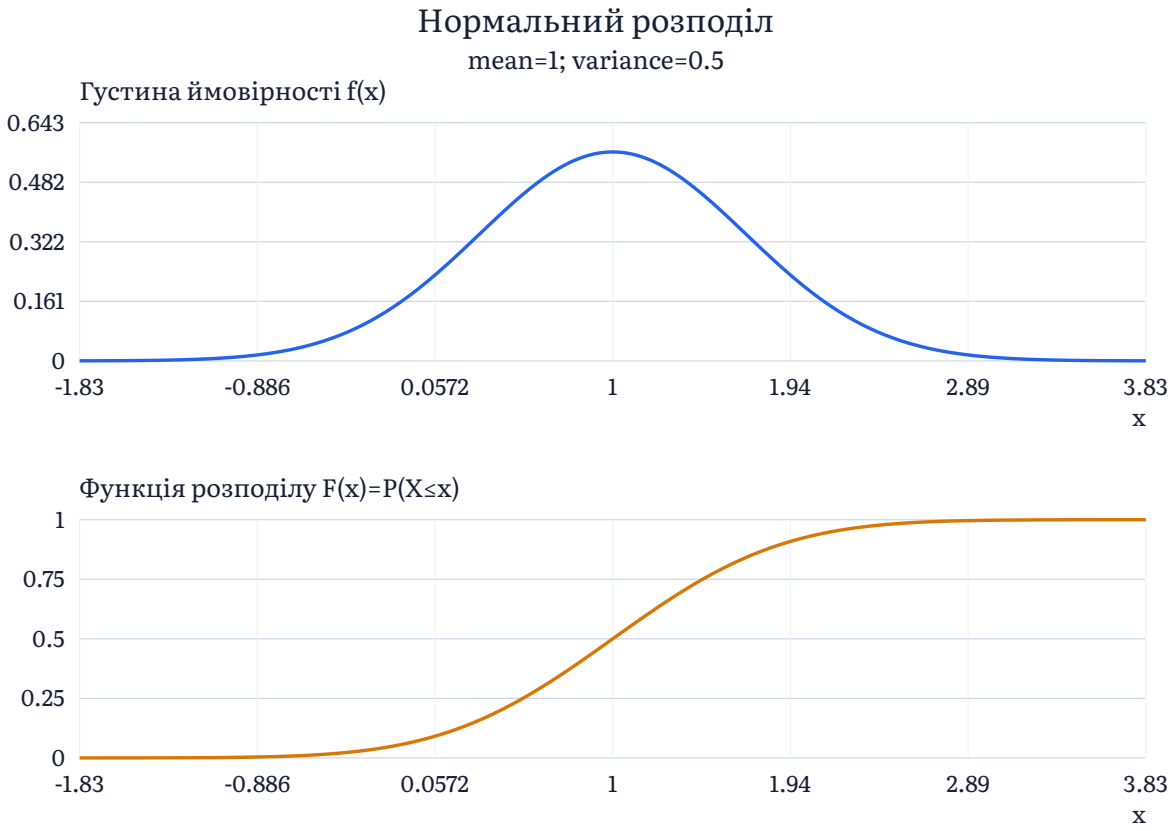



Рисунок 5 — Нормальний розподіл із μ=1 та $\sigma^2$=0.5, показаний на μ±4σ

Показаний інтервал містить частку 0.99993666 імовірності, тобто приблизно 99.9937%. Справжня підтримка розподілу — вся числова вісь.

### 5.6 Розподіл Парето

Густина визначається формулою $f(x) = \alpha \frac{x_0^\alpha}{x^{\alpha+1}}$ для $x \geq x_0$, і нулем для $x < x_0$. У формулі методички напрямок нерівності надруковано навпаки; для додатних $x_0$ та α нормований розподіл Парето має підтримку саме від $x_0$ до нескінченності.

Для $\alpha=x_0=1$ отримано $f(x) = \frac{1}{x^2}$ та $F(x) = 1 - \frac{1}{x}$ при $x \geq 1$. На рисунку x=0..10 показано 90% імовірності; праворуч від 10 лишається ще 10%. Густина швидко спадає біля нижньої межі, але довгий хвіст залишається істотним.

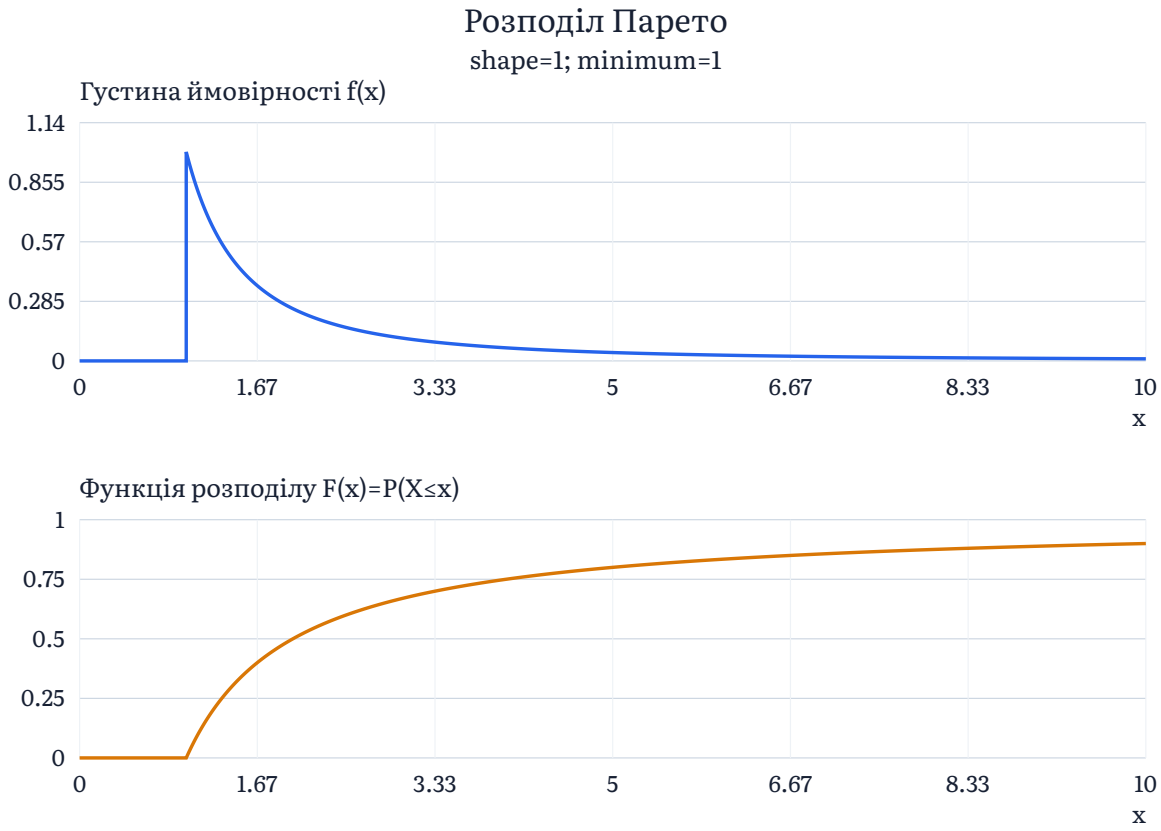



Рисунок 6 — Парето $\alpha=x_0=1$: спадна густина та CDF, яка повільно наближається до 1

За $\alpha=1$ сподівання невід'ємної величини нескінченне: інтеграл $\int_1^\infty \frac{x}{x^2}\,dx$ розбігається. Дисперсія не існує як скінченне число. Формули $E(X)=\alpha\frac{x_0}{\alpha-1}$ та $D(X)=\alpha\frac{x_0^2}{(\alpha-1)^2(\alpha-2)}$ дозволено застосовувати лише за $\alpha>1$ та $\alpha>2$ відповідно. Не можна підставляти $\alpha=1$ у ці формули як у звичайний числовий вираз.

## 5.7 Розподіл Стьюдента

$$f(x)=\frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\Gamma\left(\frac{\nu}{2}\right)\sqrt{\nu\pi}}\left(1+\frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}. \tag{4}$$

Для $\nu=1$ маємо стандартний розподіл Коші: $f(x)=\frac{1}{\pi(1+x^2)}$, $F(x)=\frac{1}{2}+\frac{\arctan(x)}{\pi}$. Він симетричний відносно 0; його медіана та мода дорівнюють 0, але математичне сподівання не існує. Симетричне головне значення інтеграла 0 не є математичним сподіванням, бо позитивна й негативна частини інтеграла не є скінченними. Скінченної дисперсії також немає.

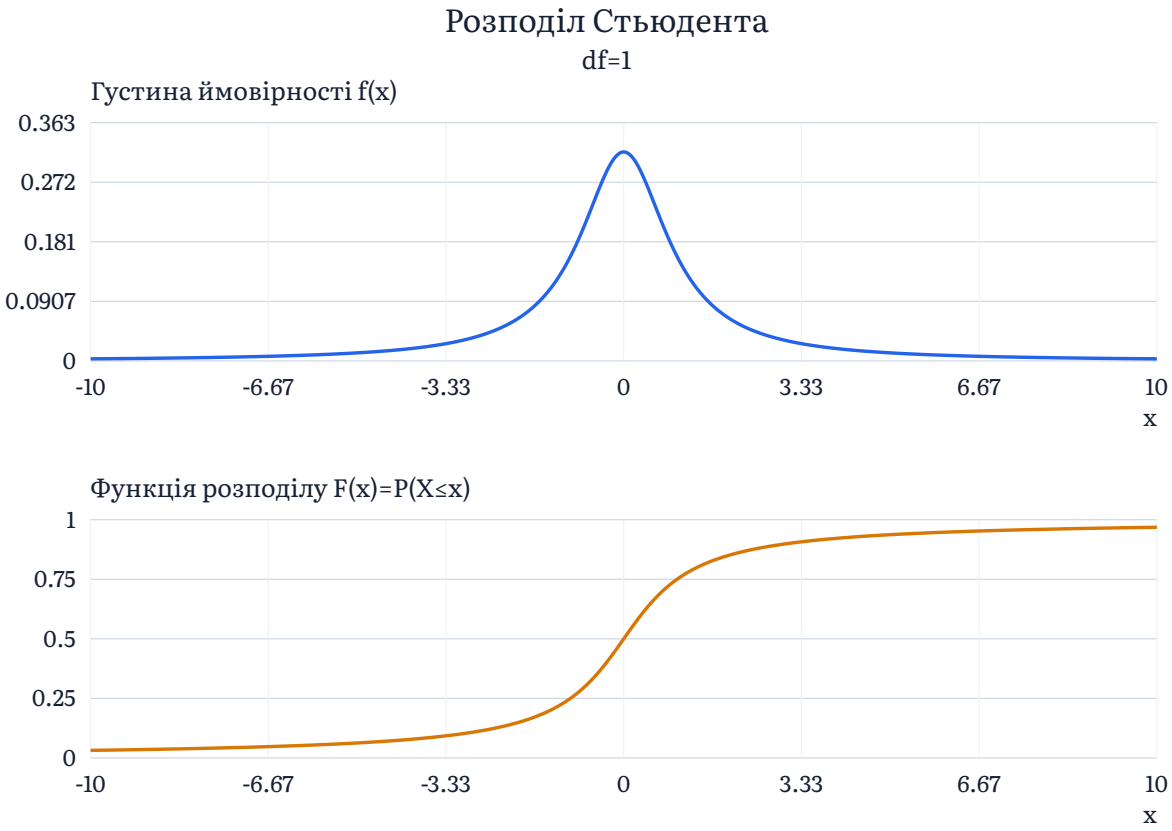



Рисунок 7 — Стьюдента ν=1 — Коші: симетрична густина з важкими хвостами

Інтервал [−10,10] містить частку 0.93654897 імовірності, приблизно 93.6549%. Решта 6.3451% розподілена між двома хвостами.

## 5.8 Порівняння хвостів

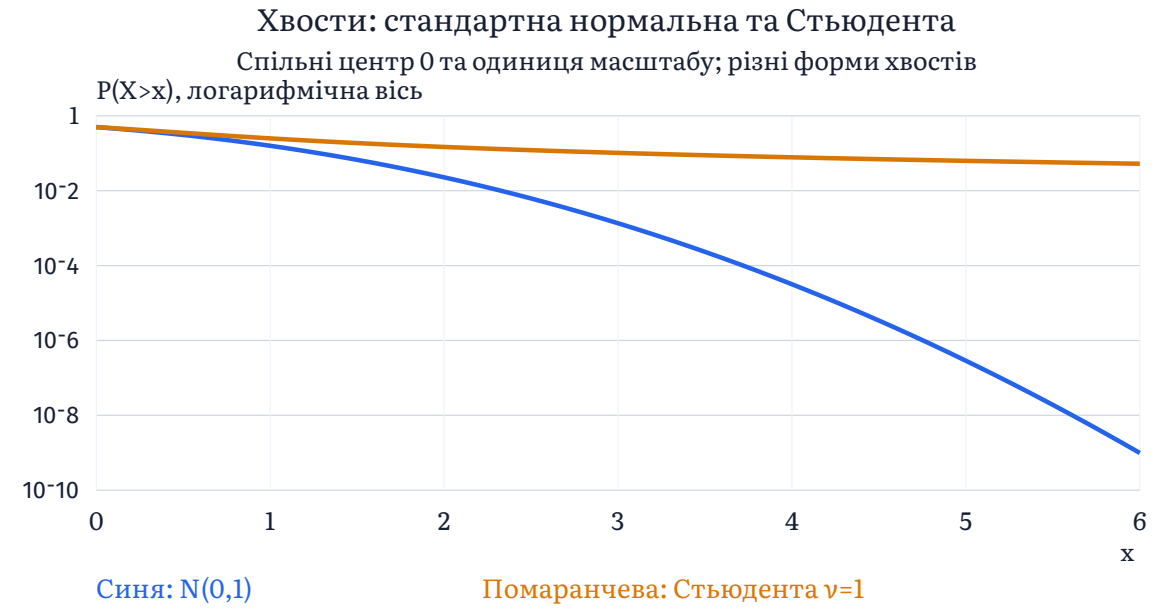



Рисунок 8 — Праві хвости стандартної нормальної та Стьюдента ν=1 на логарифмічній шкалі імовірності

Для цього допоміжного рисунка нормальний закон стандартизовано до N(0,1), щоб обидва розподіли мали центр 0 й одиницю масштабу. Це окреме порівняння форм, а не заміна заданої нормальної N(1,0.5). При x=3 хвіст нормального становить приблизно 0.001350, а Коші — 0.102416; при x=6 — приблизно 0.000000000987 і 0.052568 відповідно. Логарифмічна вісь показує, що різниця зростає на багато порядків із віддаленням від центра. Для Коші одиниця масштабу не є стандартним відхиленням, оскільки його дисперсія не існує.

## 6 Підсумкова таблиця

| **Розподіл** | **E(X)** | **D(X)** | **Показана маса** |
|---|---|---|---|
| Розподіл Бернуллі | 0.5 | 0.25 | 1 |
| Біноміальний розподіл | 1.5 | 0.75 | 1 |
| Розподіл Пуассона | 11 | 11 | 0.078614 |
| Рівномірний розподіл | 0 | 0.333333 | 1 |
| Нормальний розподіл | 1 | 0.5 | 0.999937 |
| Розподіл Парето | +∞ | не існує як скінченна | 0.9 |
| Розподіл Стьюдента | не існує | не існує як скінченна | 0.936549 |

Таблиця 2 — Теоретичні моменти та імовірнісна маса показаних діапазонів

Показана маса визначається точними формулами CDF, а не числовим інтегруванням ламаної SVG. Усі моменти в таблиці стосуються повних законів розподілу, а не лише зображених фрагментів.

## 7 Висновки

Я реалізував сім розподілів для N=1 та побудував PMF/PDF разом із CDF. Бернуллі й біноміальний мають скінченну підтримку, Пуассон — нескінченну дискретну, а чотири інші — неперервну. Найсуттєвіша особливість мого варіанта — важкі хвости Парето та Коші: наявність коректної густини не гарантує скінченних момен-

тів. Також заданий діапазон Пуассона охоплює лише малу частину ймовірності, тому його рисунок слід інтерпретувати як фрагмент повного розподілу. Програма дозволяє змінювати параметри, а всі рисунки та дані відтворюються одним запуском.

## 8 Використані джерела


1. Методичні вказівки до лабораторної роботи № 1 «Розподіли випадкових величин — неперервні і дискретні», `materials/Lab1.pdf`.
2. Катренко А. В., Пасічник В. В., Пасько В. П. Теорія прийняття рішень. Київ: BHV, 2009. 448 с.
3. Моклянчук М. П. Лекції з теорії вибору та прийняття рішень. Київ, 2007. 258 с.


## 9 Додаток А. Обчислення розподілів

```
"""Сім дискретних/неперервних розподілів; параметри CLI або JSON."""

import argparse
import csv
import json
import math
from pathlib import Path
from charts import generate_charts, make_dashboard

LAB_ROOT = Path(__file__).resolve().parent.parent


def parameters(number):
    return {"student_number": number,
            "bernoulli": {"p": 1 / (number + 1)},
            "binomial": {"p": 1 / (number + 1), "n": number + 2},
            "poisson": {"rate": number + 10, "kmax": number + 5},
            "uniform": {"a": -number, "b": number},
            "normal": {"mean": number, "variance": number / 2},
            "pareto": {"shape": number, "minimum": number},
            "student": {"df": number}}


def beta_fraction(a, b, x):
    """Ланцюговий дріб неповної бета-функції (модифікований алгоритм
Ленца)."""
    tiny = 1e-300
    c = 1.0
    d = 1 - (a + b) * x / (a + 1)
    d = 1 / (d if abs(d) > tiny else tiny)
    h = d
    for m in range(1, 301):
        even = m * (b - m) * x / ((a + 2 * m - 1) * (a + 2 * m))
```

```
        odd = -(a + m) * (a + b + m) * x / ((a + 2 * m) * (a + 2 * m +
1))
        for term in (even, odd):
            d = 1 + term * d
            c = 1 + term / c
            d = 1 / (d if abs(d) > tiny else tiny)
            c = c if abs(c) > tiny else tiny
            delta = d * c
            h *= delta
        if abs(delta - 1) < 3e-14:
            return h
    raise ArithmeticError("Неповна бета-функція не збіглася.")


def regularized_beta(x, a, b):
    if x <= 0:
        return 0.0
    if x >= 1:
        return 1.0
    factor = math.exp(math.lgamma(a + b) - math.lgamma(a) -
math.lgamma(b)
                      + a * math.log(x) + b * math.log1p(-x))
    if x < (a + 1) / (a + b + 2):
        return factor * beta_fraction(a, b, x) / a
    return 1 - factor * beta_fraction(b, a, 1 - x) / b


def student_pdf(x, df):
    coefficient = math.exp(math.lgamma((df + 1) / 2) -
math.lgamma(df / 2)) / math.sqrt(df * math.pi)
    return coefficient * (1 + x * x / df) ** (-(df + 1) / 2)


def student_cdf(x, df):
    if df == 1:
        return 0.5 + math.atan(x) / math.pi
    probability = 0.5 * regularized_beta(df / (df + x * x), df / 2,
0.5)
    return probability if x < 0 else 1 - probability


def continuous(name, label, params, start, stop, pdf, cdf, mean,
variance, extra_points=()):
    xs = sorted(set([start + (stop - start) * i / 800 for i in
range(801)] + list(extra_points)))
    return {"key": name, "label": label, "kind": "continuous",
"parameters": params,
            "rows": [{"x": x, "value": pdf(x), "cdf": cdf(x)} for x in
xs],
            "mean": mean, "variance": variance,
            "visible_mass": cdf(stop) - cdf(start)}


def discrete(name, label, params, xs, pmf, mean, variance):
    total = 0
```

```
    rows = []
    for x in xs:
        value = pmf(x)
        total += value
        rows.append({"x": x, "value": value, "cdf": total})
    return {"key": name, "label": label, "kind": "discrete",
"parameters": params,
            "rows": rows, "mean": mean, "variance": variance,
"visible_mass": total}


def solve(config):
    number = config["student_number"]
    if not isinstance(number, int) or number < 1:
        raise ValueError("N має бути додатним цілим.")
    bern, bino, pois = (config[key] for key in ("bernoulli",
"binomial", "poisson"))
    unif, norm, par, stu = (config[key] for key in ("uniform",
"normal", "pareto", "student"))
    if not 0 < bern["p"] < 1 or not 0 < bino["p"] < 1:
        raise ValueError("Ймовірності повинні належати (0,1).")
    if not isinstance(bino["n"], int) or bino["n"] < 1 or not
isinstance(pois["kmax"], int) or pois["kmax"] < 0:
        raise ValueError("n — додатне ціле; kmax — невід'ємне ціле.")
    if pois["rate"] ⩽ 0 or unif["a"] ⩾ unif["b"] or norm["variance"]
⩽ 0 or min(par.values()) ⩽ 0 or stu["df"] ⩽ 0:
        raise ValueError("Некоректні параметри розподілу.")
    p, n, rate = bino["p"], bino["n"], pois["rate"]
    distributions = [
        discrete("bernoulli", "Розподіл Бернуллі", bern, range(2),
                 lambda x: bern["p"] if x == 1 else 1 - bern["p"],
bern["p"], bern["p"] * (1 - bern["p"])),
        discrete("binomial", "Біноміальний розподіл", bino, range(n +
1),
                 lambda x: math.comb(n, x) * p ** x * (1 - p) ** (n -
x), n * p, n * p * (1 - p)),
        discrete("poisson", "Розподіл Пуассона", pois,
range(pois["kmax"] + 1),
                 lambda x: math.exp(-rate + x * math.log(rate) -
math.lgamma(x + 1)), rate, rate),
    ]
    a, b = unif["a"], unif["b"]
    # Додаткові точки обабіч меж відображають стрибки густини без
згладжування.
    gap = (b - a) * 1e-8
    distributions.append(continuous("uniform", "Рівномірний розподіл",
unif, a - (b - a) / 4, b + (b - a) / 4,
                                    lambda x: 1 / (b - a) if a ⩽ x ⩽
b else 0,
                                    lambda x: max(0, min(1, (x - a) /
(b - a))),
                                    (a + b) / 2, (b - a) ** 2 / 12, (a
- gap, a, b, b + gap)))
    mean, variance = norm["mean"], norm["variance"]
    sigma = math.sqrt(variance)
```

```
    distributions.append(continuous("normal", "Нормальний розподіл",
norm, mean - 4 * sigma, mean + 4 * sigma,
                                    lambda x: math.exp(-(x - mean) **
2 / (2 * variance)) / math.sqrt(2 * math.pi * variance),
                                    lambda x: (1 + math.erf((x -
mean) / math.sqrt(2 * variance))) / 2,
                                    mean, variance))
    shape, minimum = par["shape"], par["minimum"]
    distributions.append(continuous("pareto", "Розподіл Парето", par,
0, 10 * minimum,
                                    lambda x: shape * minimum **
shape / x ** (shape + 1) if x >= minimum else 0,
                                    lambda x: 1 - (minimum / x) **
shape if x >= minimum else 0,
                                    shape * minimum / (shape - 1) if
shape > 1 else "+∞",
                                    shape * minimum ** 2 / ((shape - 1)
** 2 * (shape - 2)) if shape > 2 else "не існує як скінченна",
                                    (minimum - minimum * 1e-8,
minimum)))
    df = stu["df"]
    distributions.append(continuous("student", "Розподіл Стьюдента",
stu, -10, 10,
                                    lambda x: student_pdf(x, df),
lambda x: student_cdf(x, df),
                                    0 if df > 1 else "не існує",
                                    df / (df - 2) if df > 2 else "не
існує як скінченна"))
    tails = [{"x": x / 10, "normal_survival": 0.5 * math.erfc(x / 10 /
math.sqrt(2)),
              "student_survival": 1 - student_cdf(x / 10, df)} for x
in range(61)]
    return {"student_number": number, "config": config,
"distributions": distributions,
            "tails": tails, "tail_comparison": "Стандартна N(0,1) і
Стьюдента з ν=N"}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--student-number", type=int, default=1)
    parser.add_argument("--input", type=Path, help="Параметри всіх
розподілів у JSON")
    parser.add_argument("--output", type=Path, default=LAB_ROOT /
"data")
    parser.add_argument("--figures-dir", type=Path, default=LAB_ROOT /
"figures")
    args = parser.parse_args()
    try:
        config = json.loads(args.input.read_text(encoding="utf-8")) if
args.input else parameters(args.student_number)
        result = solve(config)
        args.output.mkdir(parents=True, exist_ok=True)
        args.figures_dir.mkdir(parents=True, exist_ok=True)
        (args.output / "results.json").write_text(json.dumps(result,
```

```
ensure_ascii=False, indent=2, allow_nan=False) + "\n",
encoding="utf-8")
        for distribution in result["distributions"]:
            with (args.output / (distribution["key"] +
".csv")).open("w", newline="", encoding="utf-8") as stream:
                writer = csv.DictWriter(stream, fieldnames=("x",
"value", "cdf"))
                writer.writeheader()
                writer.writerows(distribution["rows"])
        generate_charts(result, args.figures_dir)
        make_dashboard(result, LAB_ROOT / "index.html")
    except (ValueError, KeyError, TypeError, OSError, ArithmeticError)
as exc:
        parser.exit(2, f"Помилка: {exc}\n")
    for distribution in result["distributions"]:
        print(f'{distribution["label"]}: E={distribution["mean"]};
D={distribution["variance"]}; '
              f'показана маса={distribution["visible_mass"]:.9f}')


if __name__ == "__main__":
    main()
```

## 10 Додаток Б. Побудова рисунків

```
"""SVG-графіки PMF/PDF та CDF без зовнішніх бібліотек."""

import html
import json
import math
from pathlib import Path

COLORS = ("#2563eb", "#d97706")


def text(x, y, label, anchor="start", size=21, fill="#172033"):
    return f'<text x="{x:.2f}" y="{y:.2f}" text-anchor="{anchor}"
font-size="{size}" fill="{fill}">{html.escape(str(label))}</text>'


def line(x1, y1, x2, y2, color="#cbd5e1", width=1):
    return f'<line x1="{x1:.2f}" y1="{y1:.2f}" x2="{x2:.2f}"
y2="{y2:.2f}" stroke="{color}" stroke-width="{width}"/>'


def save(path, pieces, height=730):
    header = f'<svg xmlns="http://www.w3.org/2000/svg" width="1100"
height="{height}" viewBox="0 0 1100 {height}"><rect width="100%"
height="100%" fill="white"/><g font-family="Literata, DejaVu Sans,
serif">'
    path.write_text(header + "\n" + "\n".join(pieces) + "</g></svg>",
encoding="utf-8")
```

```
def axes(start, stop, ymax, top, title):
    left, width, height = 95, 940, 210
    xx = lambda x: left + width * (x - start) / (stop - start)
    yy = lambda y: top + height * (1 - y / ymax)
    pieces = [text(left, top - 20, title, size=22)]
    for i in range(5):
        y = ymax * i / 4
        pieces.extend([line(left, yy(y), left + width, yy(y)),
text(left - 13, yy(y) + 7, f"{y:.3g}", "end", 20)])
    for i in range(7):
        x = start + (stop - start) * i / 6
        pieces.extend([line(xx(x), top, xx(x), top + height,
"#eef2f6"), text(xx(x), top + height + 30, f"{x:.3g}", "middle", 20)])
    pieces.append(text(left + width, top + height + 57, "x", "end",
21))
    return pieces, xx, yy


def generate_charts(results, folder):
    folder = Path(folder)
    for distribution in results["distributions"]:
        rows = distribution["rows"]
        discrete = distribution["kind"] == "discrete"
        start = rows[0]["x"] - 0.5 if discrete else rows[0]["x"]
        stop = rows[-1]["x"] + 0.5 if discrete else rows[-1]["x"]
        ymax = max(row["value"] for row in rows) * 1.14
        params = "; ".join(f"{key}={value:g}" for key, value in
distribution["parameters"].items())
        pieces = [text(550, 35, distribution["label"], "middle", 28),
text(550, 68, params, "middle", 22)]
        panel, xx, yy = axes(start, stop, ymax, 115, "Імовірність
P(X=k)" if discrete else "Густина ймовірності f(x)")
        pieces.extend(panel)
        if discrete:
            width = min(80, (xx(1) - xx(0)) * 0.6)
            for row in rows:
                x, value = xx(row["x"]), row["value"]
                pieces.append(f'<rect x="{x-width/2:.2f}"
y="{yy(value):.2f}" width="{width:.2f}" height="{yy(0)-yy(value):.2f}"
fill="{COLORS[0]}"/>')
                pieces.append(text(x, yy(value) - 9, f"{value:.4f}",
"middle", 19))
        else:
            points = " ".join(f'{xx(row["x"]):.2f},
{yy(row["value"]):.2f}' for row in rows)
            pieces.append(f'<polyline points="{points}" fill="none"
stroke="{COLORS[0]}" stroke-width="3"/>')
        panel, xx, yy = axes(start, stop, 1, 465, "Функція розподілу
F(x)=P(X≤x)")
        pieces.extend(panel)
        if discrete:
            previous, previous_x = 0, start
            for row in rows:
                x, cdf = row["x"], row["cdf"]
                pieces.append(line(xx(previous_x), yy(previous),
```

```
xx(x), yy(previous), COLORS[1], 3))
                pieces.append(f'<circle cx="{xx(x):.2f}"
cy="{yy(previous):.2f}" r="4" fill="white" stroke="{COLORS[1]}"
stroke-width="2"/>')
                pieces.append(f'<circle cx="{xx(x):.2f}"
cy="{yy(cdf):.2f}" r="4" fill="{COLORS[1]}"/>')
                previous, previous_x = cdf, x
            pieces.append(line(xx(previous_x), yy(previous), xx(stop),
yy(previous), COLORS[1], 3))
        else:
            points = " ".join(f'{xx(row["x"]):.2f},
{yy(row["cdf"]):.2f}' for row in rows)
            pieces.append(f'<polyline points="{points}" fill="none"
stroke="{COLORS[1]}" stroke-width="3"/>')
        save(folder / (distribution["key"] + ".svg"), pieces, 745)
    rows = results["tails"]
    pieces = [text(550, 35, "Хвости: стандартна нормальна та
Стьюдента", "middle", 27),
              text(550, 70, "Спільні центр 0 та одиниця масштабу;
різні форми хвостів", "middle", 22)]
    left, top, width, height = 100, 110, 920, 330
    xx = lambda x: left + width * x / 6
    yy = lambda p: top + height * (-math.log10(max(p, 1e-10))) / 10
    for power in range(0, 11, 2):
        y = top + height * power / 10
        pieces.extend([line(left, y, left + width, y), text(left - 15,
y + 7, f"10⁻{power}" if power else "1", "end", 21)])
    for x in range(7):
        pieces.extend([line(xx(x), top, xx(x), top + height,
"#eef2f6"), text(xx(x), top + height + 35, x, "middle", 22)])
    for index, key in enumerate(("normal_survival",
"student_survival")):
        points = " ".join(f'{xx(row["x"]):.2f},{yy(row[key]):.2f}' for
row in rows)
        pieces.append(f'<polyline points="{points}" fill="none"
stroke="{COLORS[index]}" stroke-width="4"/>')
    pieces.extend([text(100, 95, "P(X>x), логарифмічна вісь",
size=22), text(1015, 502, "x", "end"),
                   text(100, 535, "Синя: N(0,1)", size=23,
fill=COLORS[0]),
                   text(440, 535, f'Помаранчева: Стьюдента
v={results["config"]["student"]["df"]}', size=23, fill=COLORS[1])])
    save(folder / "tail-comparison.svg", pieces, 575)


def make_dashboard(results, destination):
    parts = ['<!doctype html><html lang="uk"><meta
charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1">',
             '<title>Лабораторна 1 — розподіли</title><style>body{max-
width:1000px;margin:32px auto;padding:0 24px;font:18px/1.6
Literata,serif;color:#172033}img{width:100%}figure{margin:24px 0}
figcaption{text-align:center}table{border-
collapse:collapse;width:100%}td,th{border:1px solid
#cbd5e1;padding:10px;text-align:left}</style>',
```

```
            f'<h1>Розподіли випадкових величин</h1><p>Біжик Юрій
Володимирович · ФЕІ-34с · N={results["student_number"]}</p>',
            '<p>Теоретичні PMF/PDF і CDF; випадкові вибірки не
генеруються.</p>']
    for i, distribution in enumerate(results["distributions"], 1):
        label = html.escape(distribution["label"])
        parts.append(f'<figure><img src="figures/
{distribution["key"]}.svg" alt="{label}"><figcaption>Рисунок {i} —
{label}: PMF/PDF та CDF</figcaption></figure>')
    parts.append('<figure><img src="figures/tail-comparison.svg"
alt="Порівняння хвостів"><figcaption>Рисунок 8 — Порівняння правих
хвостів</figcaption></figure>')
    parts.append('<p>Розподіл Пуассона зображений лише до заданого
kmax без перенормування. У Парето α=1 сподівання нескінченне, у
Стьюдента ν=1 воно не існує.</p></html>')
    destination.write_text("\n".join(parts), encoding="utf-8")


if __name__ == "__main__":
    root = Path(__file__).resolve().parent.parent
    generate_charts(json.loads((root / "data/
results.json").read_text(encoding="utf-8")), root / "figures")
```